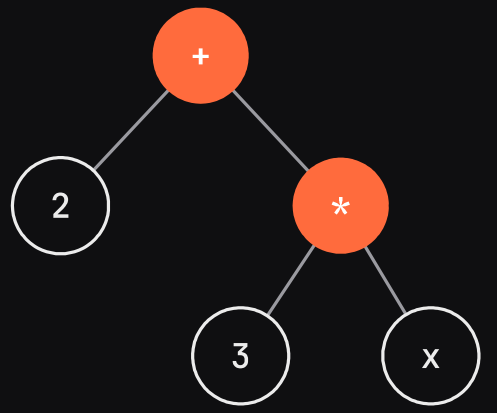
CHARLA INTRODUCTORIA

Diseñar antes de codificar

Patrones de Diseño y Arquitectura para Proyectos Escalables



Alvaro Josué Morales Rodríguez

2 + 3 * x

QUIÉN PRESENTA

Perfil del conferencista

Alvaro Josué Morales Rodríguez

Full-Stack Developer & Cloud Administrator

GitHub github.com/ELFBIBv

LinkedIn linkedin.com/in/alvaro-josue-morales

Correo jm54336720@gmail.com

Ing. en Ciencias y Sistemas · USAC — 9.º semestre

Tutor líder de computación · TutoMath

HABILIDADES CLAVE

Java / Go / Python	Intermedio
React / Node	Semi-avanzado
Docker / Linux	Semi-avanzado
AWS Cloud	Intermedio
Ciberseguridad	Intermedio

PROYECTOS DESTACADOS

01

Infraestructura cloud

EC2, S3, API Gateway y DynamoDB en producción.

02

Automatización

Bot de Discord para monitoreo y comandos remotos.

03

IA integrada

Chatbot con entrada de audio y documentos.

04

Data Warehouse

Esquema de constelación diseñado antes de los ETL.

ANTES DE EMPEZAR

Una historia que tal vez conocen

Semana 1	`Main.java` funciona. Todo bien.
Semana 2	"Solo agrego otro `if`..."
Semana 3	`Main.java` ya va por las 1,500 líneas.
Semana 4	Arreglo un bug y aparecen dos.
Semana 5	"Mejor lo hago otra vez desde cero."

```
// Main.java, línea 1,342
if (opcion == 1) {
    if (tipo.equals("admin")) {
        if (lista.size() > 0) {
            for (int i = 0; i < lista.size(); i++) {
                if (lista.get(i).getEstado() == 3) {
                    // ¿qué significa 3?
                    // ...
                }
            }
        }
    } else if (tipo.equals("usuario")) {
        // copiado y pegado del bloque de arriba
```

Escribir código es la parte fácil. Lo difícil es cambiarlo sin romper lo que ya funciona.

01 · DISEÑAR ANTES DE CODIFICAR

¿Qué significa diseñar?

100×

más caro puede salir corregir en producción un error que en el diseño se arreglaba borrando una flecha.

Barry Boehm, *Software Engineering Economics* (1981)

Diseñar es

- Pensar qué responsabilidades existen
- Dividir el problema en partes (clases)
- Decidir cómo se comunican esas partes

Diseñar no es

- Escribir un documento de 100 páginas
- Hacer el diagrama UML perfecto
- Pasar meses sin programar

Con papel, lápiz y diez minutos alcanza.

01 · DISEÑAR ANTES DE CODIFICAR

¿Qué es un proyecto escalable?

Uno que puede crecer sin derrumbarse. ¿Crecer en qué?

01

Funcionalidades

Agregar cosas nuevas sin romper las que ya están.

02

Equipo

Que varias personas trabajen en el mismo código sin pisarse.

03

Usuarios

Más usuarios, más datos, más peticiones por segundo.

SEÑALES DE ALERTA

- Cada cambio rompe otra cosa
- Solo una persona entiende el código
- Para agregar algo hay que copiar y pegar algo ya existente

> Los **patrones de diseño** ayudan sobre todo con los dos primeros puntos. Los de **arquitectura**, con los tres.

02 · PATRONES

¿Qué es un patrón?

Una solución probada a un problema que aparece una y otra vez.

Es

- Una idea, una plantilla de solución
- Un vocabulario que comparten los programadores
- Independiente del lenguaje

No es

- Una librería que se descarga
- Código para copiar y pegar
- Algo que hay que usar porque sí

Como una receta de pepián: la receta no es el pepián. Cada quien lo cocina con lo que tiene, pero todos reconocen el platillo.

02 · PATRONES

¿De dónde vienen?

1977

A Pattern Language

Christopher Alexander, un arquitecto de edificios.

1987

Beck y Cunningham

Llevan la idea al software.

1994

Design Patterns

El "Gang of Four" publica 23 patrones de diseño.

1996

POSA

Patrones de arquitectura.

Hoy

En todos lados

Java, Js, Ts, Android, React, Fortran xd...

> Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno, y luego describe el núcleo de la solución a ese problema.
>
> Christopher Alexander, 1977

02 · PATRONES

Tipos de patrones

Arquitectura
todo el sistema

Capas · Cliente-Servidor · MVC · **Pipes & Filters** HOY · Microservicios

El plano de la ciudad: cómo se reparten y se comunican las piezas grandes.

Diseño
unas pocas clases
el plano de una casa

5
Creacionales
cómo se crean los objetos

7
Estructurales
cómo se combinan las piezas

11
De comportamiento
cómo se reparten el trabajo

Son los 23 del libro del Gang of Four. En las siguientes tres diapositivas están todos.

Creacionales

5

DE 23

¿Quién hace el `new`, y cuándo?



HOY 01

Factory Method

Un método decide qué objeto crear, en vez de usar new

`Calendar.getInstance()`



02

Abstract Factory

Crea familias de objetos que combinan entre sí

tema claro y tema oscuro



03

Builder

Arma un objeto complicado paso a paso

`StringBuilder`



04

Prototype

Crea un objeto nuevo copiando uno que ya existe

`objeto.clone()`



05

Singleton

Garantiza una sola instancia en todo el programa

`Runtime.getRuntime()`

> Singleton es el más conocido y también el más abusado: en el fondo es una variable global con otro nombre. Úsenlo con cuidado.

Estructurales

7

¿Cómo junto piezas para formar algo más grande?

DE 23



06

Adapter

Hace compatibles dos interfaces que no encajan

`Arrays.asList(arreglo)`



07

Bridge

Separa la idea de su implementación para que cambien por separado

control remoto y televisor



08

Composite

Trata igual a un elemento y a un grupo de elementos

una carpeta mide lo que suman sus archivos y subcarpetas



09

Decorator

Envuelve un objeto en otro que le agrega funciones

BufferedReader envuelve a FileReader y le agrega readLine()



10

Facade

Una sola clase sencilla que esconde un sistema complicado

tienda.comprar() llama por ti a inventario, pago y envío



11

Flyweight

Comparte los datos repetidos para ahorrar memoria

10,000 árboles de un juego comparten un solo modelo 3D



12

Proxy

Un intermediario que controla el acceso a otro objeto

carga una imagen pesada solo cuando se va a mostrar

Composite es la base del patrón Interpreter: un nodo que contiene otros nodos. Lo van a ver en el árbol de mi proyecto.

De comportamiento

11 DE 23

¿Quién hace qué, y cómo se avisan entre ellos?



13

Chain of Responsibility

Pasa la petición hasta que alguien la atiende

soporte por niveles



14

Command

Convierte una acción en un objeto

la comanda del mesero



HOY 15

Interpreter

Cada regla de una gramática se vuelve una clase

`Pattern.compile()`



16

Iterator

Recorre una colección sin saber cómo está hecha

`for (x : lista)`



17

Mediator

Un objeto central coordina a todos los demás

la torre de control



18

Memento

Guarda el estado de un objeto para volver a él

`Ctrl + Z`



19

Observer

Avisa a todos los suscritos cuando algo cambia

`addActionListener`



20

State

El objeto se comporta distinto según su estado

un semáforo



21

Strategy

Cambia el algoritmo sin tocar a quien lo usa

`Comparator`



22

Template Method

Define los pasos; las subclases rellenan algunos

una receta



23

Visitor

Agrega operaciones sin modificar las clases

recorrer un árbol

—

refactoring.guru muestra 10: deja fuera Interpreter, que sí está en el libro original.

02 · PATRONES

Ya los usan sin saberlo

La librería estándar de Java está llena de patrones.

CÓDIGO QUE YA CONOCEN	PATRÓN
`for (String s : lista) { … }`	Iterator
`boton.addActionListener(e -> …)`	Observer
`Runtime.getRuntime()`	Singleton
`Calendar.getInstance()`	Factory
`new BufferedReader(new FileReader("a.txt"))`	Decorator
`Pattern.compile("[a-z]+")`	Interpreter

02 · PATRONES

Ventajas y desventajas

A favor

- Vocabulario común con tu equipo
- Soluciones probadas durante décadas
- Agregar cosas sin romper lo que ya existe
- Código más fácil de mantener y de probar

En contra

- Sobreingeniería: un patrón donde bastaba un `if`
- Más clases; al inicio cuesta seguir el flujo
- Hay que entender la intención, no solo copiar
- "Si solo tienes un martillo, todo parece un clavo"

Un patrón mal aplicado es peor que no usar ninguno.

02 · PATRONES

Recomendaciones

KISS

Keep it simple. Mantenlo simple.

YAGNI

You aren't gonna need it. No programes para un futuro que te imaginas.

DRY

Don't repeat yourself. Cada lógica en un solo lugar.

---

1. **Primero el problema, después el patrón.** Nunca al revés.
2. **Empiecen simple** y usen un patrón cuando el problema de verdad aparezca.
3. **Dibujen antes de programar**, aunque sea en una hoja.
4. **Aprendan primero los básicos:** Factory Method, Singleton, Strategy y Observer.
5. **Practiquen en los proyectos del curso.** Son el mejor laboratorio.

03 · FACTORY METHOD

El problema

```java
public class Main {
    public static void main(String[] args) {
        String canal = args[0];   // "EMAIL", "SMS"...
        if (canal.equals("EMAIL")) {
            Email email = new Email();
            email.enviar("Tu pedido va en camino");
        } else if (canal.equals("SMS")) {
            Sms sms = new Sms();
            sms.enviar("Tu pedido va en camino");
        }
        // este if/else también está copiado en
        // aprobarPago(), enviarFactura()... 10 veces
    }
}
```

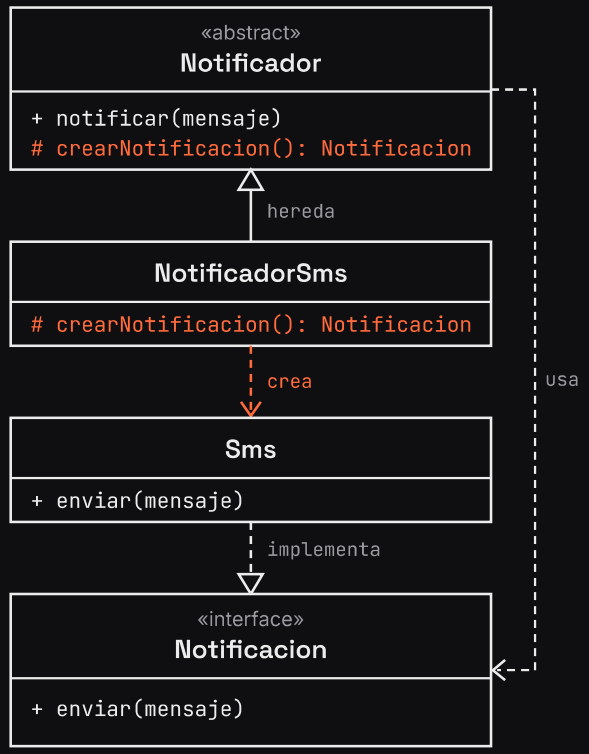
01 `Main` conoce todas las clases concretas.

02 Agregar WhatsApp obliga a tocar cada `if/else`.

03 La lógica de creación está repetida por todo el proyecto.

03 · FACTORY METHOD

La idea



> Define un método para crear objetos, pero deja que las subclases decidan qué clase crear. Gang of Four

LOS CREADORES

- `Notificador` tiene la lógica común y declara `crearNotificacion()`
- `NotificadorSms` decide qué objeto se crea

LOS PRODUCTOS

- `Notificacion` es lo que todos saben hacer: `enviar()`
- `Sms` es una forma concreta de hacerlo

Separamos usar un objeto de crearlo.

Toda la creación pasa por un solo método.

03 · FACTORY METHOD

Piénsenlo como un videojuego

Todos los niveles funcionan igual: aparece un enemigo y te ataca. Lo único que cambia es **cuál**.



TRADUCIDO A JAVA

Nivel	`Notificador`
las reglas	`notificar()`
el ?	`crearNotificacion()`
Nivel bosque	`NotificadorSms`
un enemigo	`Notificacion`
el lobo	`Sms`
el jugador	`main`

> Ese ? es el factory method. Las reglas lo usan sin saber qué bicho va a salir.

03 · FACTORY METHOD

El código

1 · EL PRODUCTO · EL ENEMIGO

```
interface Notificacion {
    void enviar(String mensaje);
}

class Sms implements Notificacion {
    public void enviar(String mensaje) {
        System.out.println("[SMS] " + mensaje);
    }
}
```

2 · EL CREADOR · LOS NIVELES

```
abstract class Notificador {
    // el factory method: el "?"
    abstract Notificacion crearNotificacion();

    void notificar(String mensaje) {   // las reglas
        crearNotificacion().enviar(mensaje);
    }
}

class NotificadorSms extends Notificador {
    Notificacion crearNotificacion() {
        return new Sms();
    }
}
```

3 · EL CLIENTE, QUE YA NO SABE QUE EXISTE SMS · EL JUGADOR

```
Notificador notificador = new NotificadorSms();
notificador.notificar("Tu pedido va en camino");   // [SMS] Tu pedido va en camino
```

03 · FACTORY METHOD

El código

1 · EL PRODUCTO · EL ENEMIGO

```
interface Notificacion { ... }

class Sms implements Notificacion { ... }

class Telegram implements Notificacion {
    public void enviar(String mensaje) {
        System.out.println("[TELEGRAM] " + mensaje);
    }
}
```

2 · EL CREADOR · LOS NIVELES

```
abstract class Notificador { ... }

class NotificadorSms extends Notificador { ... }

class NotificadorTelegram extends Notificador {
    Notificacion crearNotificacion() {
        return new Telegram();
    }
}
```

3 · EL CLIENTE, QUE YA NO SABE QUE EXISTE SMS · EL JUGADOR

```
Notificador notificador = new NotificadorTelegram();
notificador.notificar("Tu pedido va en camino");   // [TELEGRAM] Tu pedido va en camino
```

> ¿Llega Telegram? Solo se agrega código: dos clases nuevas.
> `Notificador` y `Sms` ni se tocan, y el cliente solo escoge otro nivel.

03 · FACTORY METHOD

El Main, antes y después

ANTES · DENTRO DE MAIN

```
String canal = args[0];
if (canal.equals("EMAIL")) {
    Email email = new Email();
    email.enviar("Tu pedido va en camino");
} else if (canal.equals("SMS")) {
    Sms sms = new Sms();
    sms.enviar("Tu pedido va en camino");
}
// copiado también en aprobarPago(),
// enviarFactura()... 10 veces
```

El `if` no desaparece: queda en un solo lugar. Los demás lugares solo llaman a `notificar()` y no saben que existen `Email` ni `Sms`.

DESPUÉS

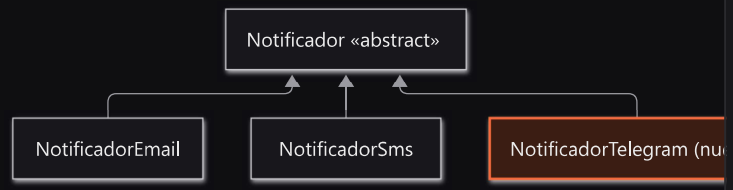
```
public class Main {
    public static void main(String[] args) {
        Notificador notificador = elegir(args[0]);
        notificador.notificar("Tu pedido va en camino");
        // aprobarPago(), enviarFactura()... hacen
        // lo mismo: notificador.notificar(...)
    }

    // el único lugar que sabe qué canales existen
    static Notificador elegir(String canal) {
        if (canal.equals("EMAIL")) {
            return new NotificadorEmail();
        } else {
            return new NotificadorSms();
        }
    }
}
```

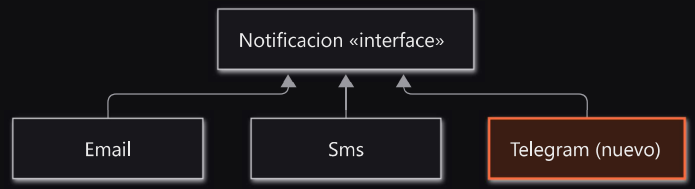
03 · FACTORY METHOD

Llega un requisito nuevo: Telegram

CREADORES



PRODUCTOS




```java
class Telegram implements Notificacion {
    public void enviar(String mensaje) {
        System.out.println("[TELEGRAM] " + mensaje);
    }
}

class NotificadorTelegram extends Notificador {
    Notificacion crearNotificacion() {
        return new Telegram();
    }
}
```

> Dos clases nuevas y una línea más en `elegir()`.
> Lo que ya funcionaba no se toca: eso es escalar.

Vale la pena cuando esperas que aparezcan tipos nuevos. Si solo hay uno, sobra.

04 · ARQUITECTURA

Pipes & Filters

Un patrón de arquitectura que no es MVC: los datos pasan por una línea de ensamblaje.



- **Filtro:** recibe datos, los transforma y los entrega. No sabe quién está antes ni después.
- **Pipe:** conecta la salida de un filtro con la entrada del siguiente.

YA LO CONOCEN: JAVA STREAMS

```
List<String> aprobados = estudiantes.stream() // fuente
    .filter(e -> e.getNota() >= 61)           // filtro 1
    .sorted(Comparator.comparing(Estudiante::getNota)
                      .reversed())            // filtro 2
    .map(Estudiante::getNombre)               // filtro 3
    .toList();                                // destino
```

> Cada etapa hace una sola cosa, así que se pueden agregar, quitar o cambiar etapas sin rediseñar todo. Lo usan los compiladores, los editores de imágenes y los servidores web.

05 · CASO REAL

Mi intérprete de OLC1

La idea: crear un lenguaje de programación propio y un IDE web donde se escribe, se ejecuta y se ve su árbol.

El reto no es el lenguaje en sí, sino cómo organizar el código para que pueda crecer: cada instrucción nueva no debería romper las anteriores.

FACTORIAL.CI

```
function int factorial(n: int) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n = n - 1);
}

function void main() {
    echo "5! = ", factorial(n = 5);
}

ejecutar main();
```

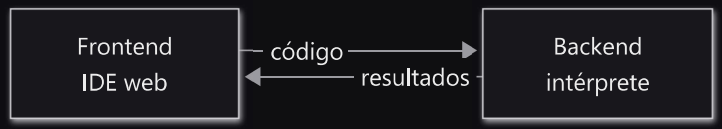
SALIDA

```
5! = 120
```

05 · CASO REAL

Cómo está armado

CLIENTE-SERVIDOR · EL NAVEGADOR MANDA EL CÓDIGO Y EL SERVIDOR LO INTERPRETA



PIPES & FILTERS · DENTRO DEL SERVIDOR, EL CÓDIGO PASA POR ETAPAS



> Cada etapa hace una sola cosa y le pasa su resultado a la siguiente. La última etapa es donde entra el patrón Interpreter.

05 · CASO REAL

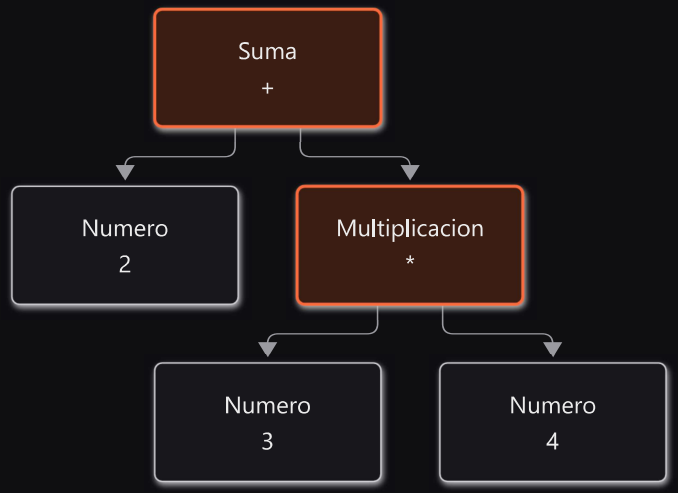
El patrón Interpreter

Cada regla del lenguaje se convierte en una clase, y cada clase sabe evaluarse a sí misma.

```
interface Expresion { int interpretar(); }

class Numero implements Expresion {      // hoja
    int valor;
    public int interpretar() { return valor; }
}

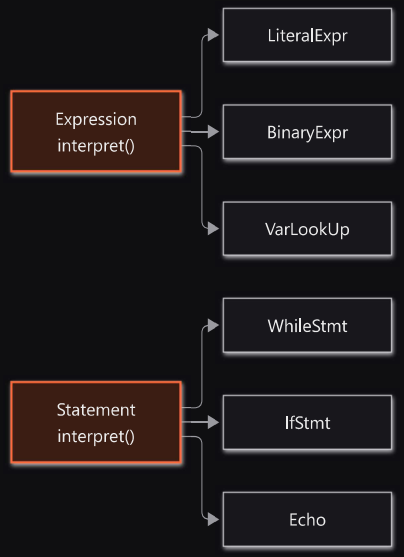
class Suma implements Expresion {        // nodo
    Expresion izq, der;
    public int interpretar() {
        return izq.interpretar() + der.interpretar();
    }
}
```



El programa `2 + 3 * 4` se guarda como un árbol (AST)

05 · CASO REAL

Cómo lo usé en el proyecto



Dos clases abstractas y una clase por cada instrucción del lenguaje

- El analizador **solo construye el árbol**; no ejecuta nada.
- Para correr el programa se llama `interpret()` en la raíz y cada nodo evalúa a sus hijos.
- Un `while` simplemente vuelve a pedirle a su condición y a su cuerpo que se interpreten.
- Agregar una instrucción nueva es **agregar una clase**, sin tocar las que ya existen.

05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.

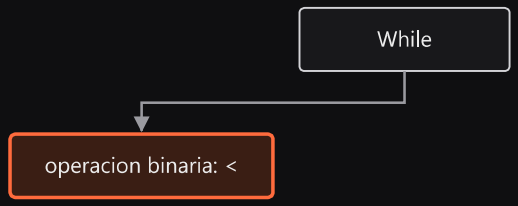


En color, las expresiones: lo que devuelve un valor

05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```



1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.

En color, las expresiones: lo que devuelve un valor

05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```



1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

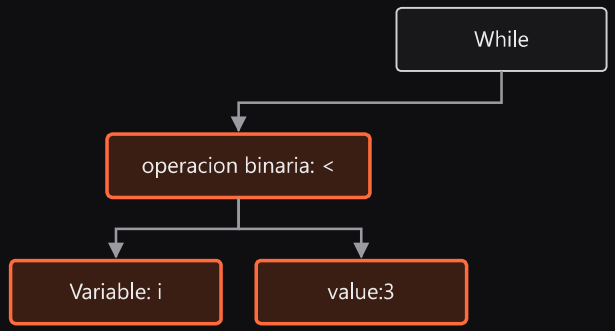
El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.

En color, las expresiones: lo que devuelve un valor

05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```



1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.

En color, las expresiones: lo que devuelve un valor

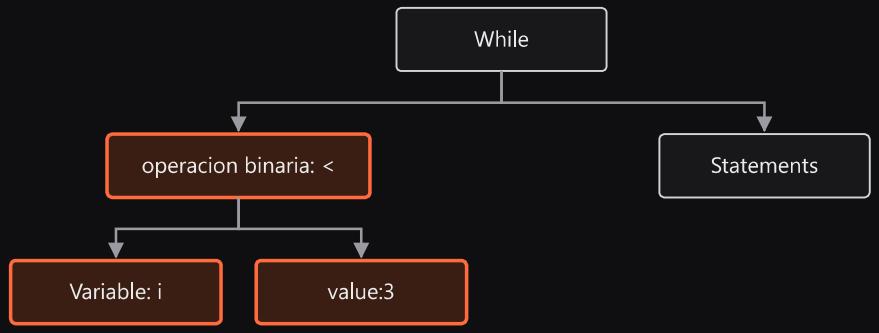
05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.



En color, las expresiones: lo que devuelve un valor

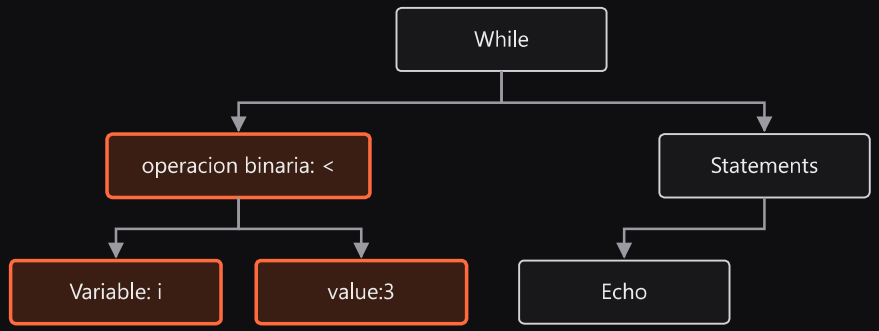
05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.



En color, las expresiones: lo que devuelve un valor

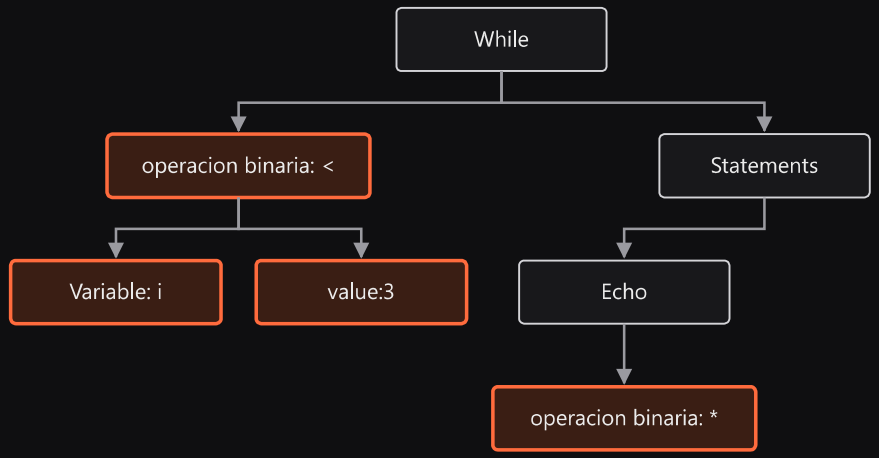
05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.



En color, las expresiones: lo que devuelve un valor

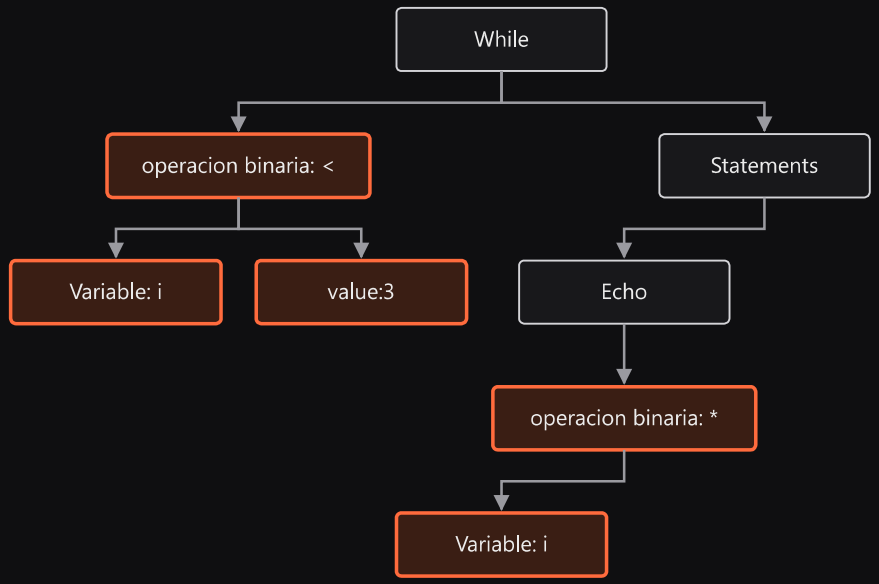
05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.



En color, las expresiones: lo que devuelve un valor

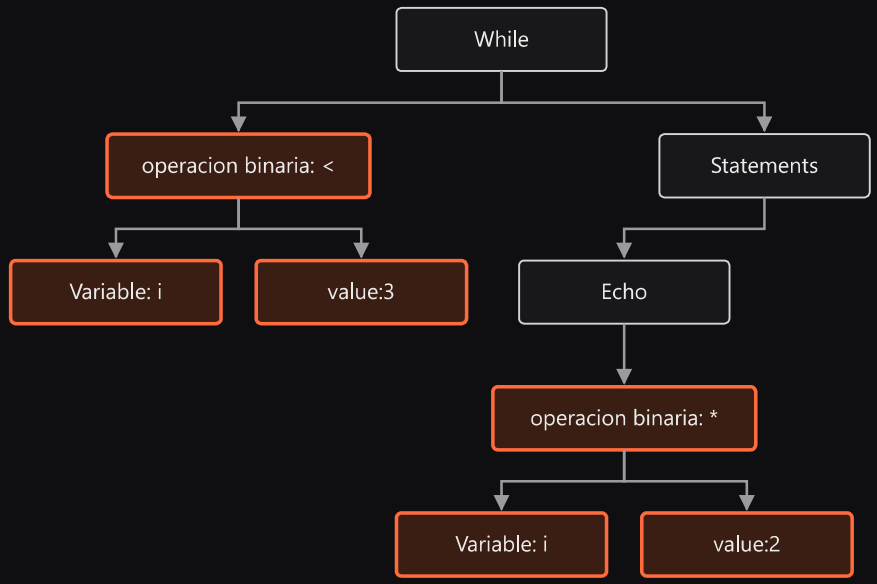
05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.



En color, las expresiones: lo que devuelve un valor

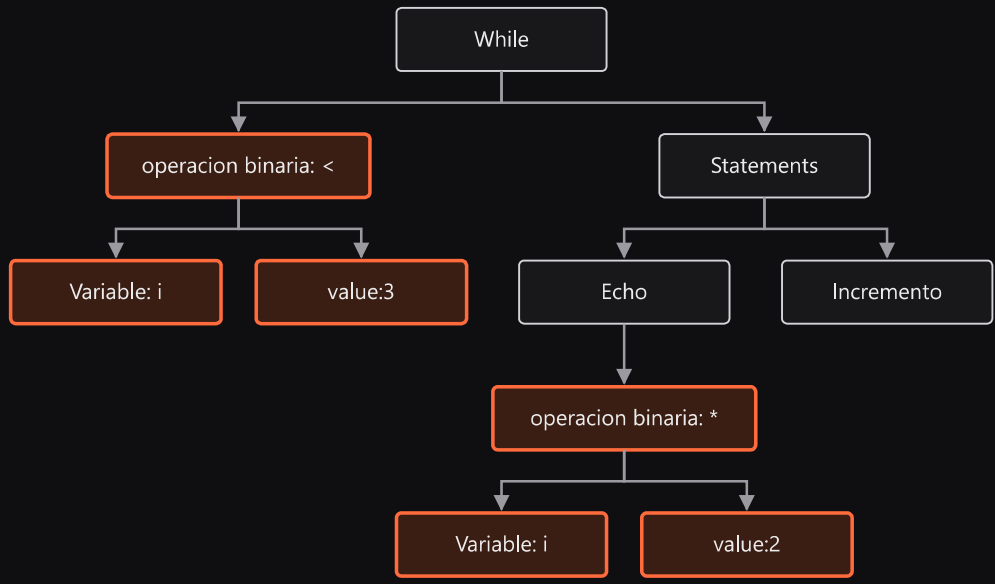
05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.



En color, las expresiones: lo que devuelve un valor

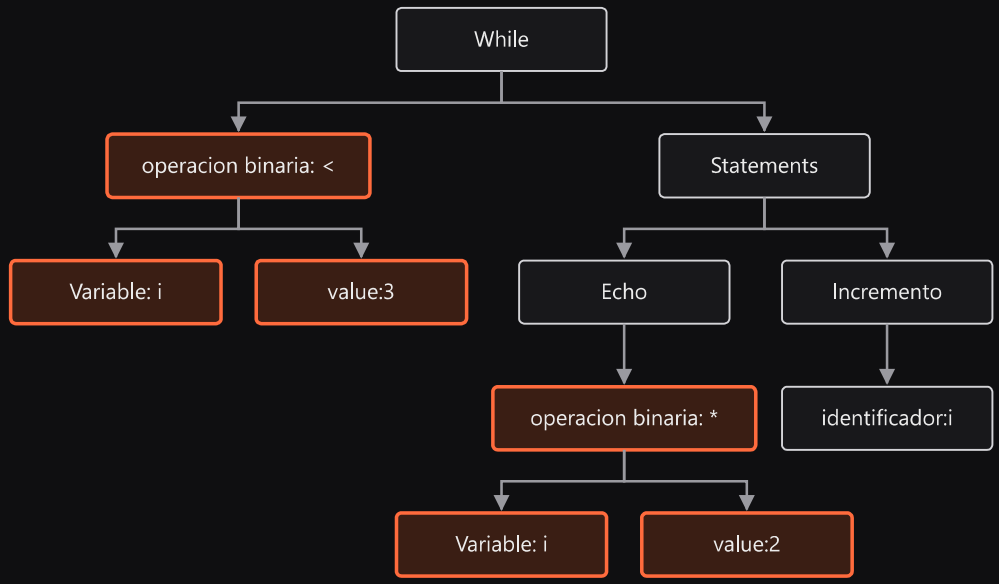
05 · CASO REAL

Un `while` como árbol

```
int i = 1;
while (i < 3) {
    echo i * 2;
    i++;
}
```

1. `While` le pide a su condición `i < 3` que se interprete.
2. Si es `true`, interpreta cada hijo de `Statements`.
3. Vuelve a preguntar la condición y repite.

El `While` no sabe comparar ni imprimir: solo coordina a sus hijos.



En color, las expresiones: lo que devuelve un valor

PARA LLEVAR

Gracias.

¿Preguntas?

01

Diseñar antes

Los errores se corrigen en papel, no en producción.

02

Factory Method

Tipos nuevos sin modificar el código que ya funciona.

03

Pipes & Filters

Etapas independientes que se agregan o cambian sin rediseñar todo.

04

Interpreter

El lenguaje crece agregando clases, sin tocar las que ya existen.

Alvaro Josué Morales Rodríguez

Para seguir: refactoring.guru/es/design-patterns